



Application de Covoiturage Écologique

Dossier de Projet — ECF

Titre Professionnel Développeur Web et Web Mobile (DWWM) — Niveau 5

Thomas Colin

Studi — Promotion 2026 · Durée de développement : 2 semaines

Sommaire

- 01 Contexte et problématique
- 02 Présentation de l'application
- 03 Architecture globale
- 04 Choix technologiques
- 05 Base de données
- 06 Développement Front-End
- 07 Développement Back-End
- 08 MongoDB — Logs d'activité
- 09 Tests
- 10 Déploiement
- 11 Difficultés et solutions
- 12 Bilan et perspectives
- 13 Conclusion

Contexte et problématique

74%

des trajets domicile-travail
sont effectués seul en voiture

20%

des émissions de CO₂ en France
proviennent du transport routier

3 rôles

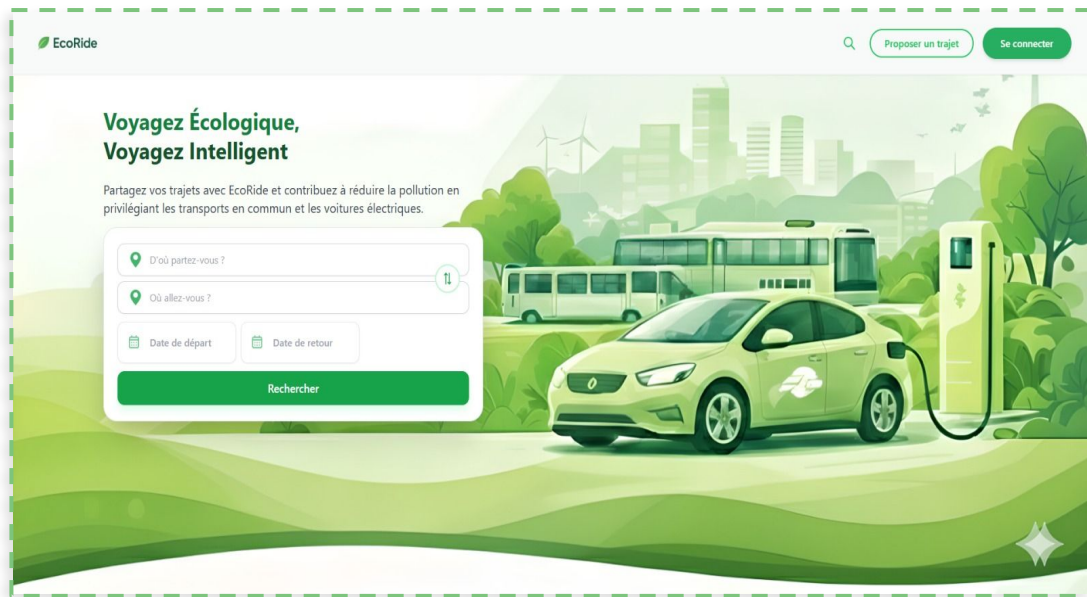
Visiteur, Utilisateur,
Administrateur

La problématique

Comment proposer une plateforme de covoiturage moderne, sécurisée et éco-responsable, couvrant l'ensemble des besoins utilisateur (recherche, réservation, gestion de compte) et les besoins d'administration (modération, statistiques, logs) ?

Objectif : Développer une application full-stack complète, de la BDD au déploiement, en solo en 3 semaines.

Présentation de l'application



Fonctionnalités clés

- Inscription / Connexion sécurisée (JWT)
- Recherche de trajets avec filtres
- Réservation & système de crédits
- Publication de trajets
- Gestion de profil et véhicule
- Espace Admin — stats & logs
- Espace Modérateur — avis & litiges

Architecture globale — Monorepo + MVP

CLIENT (Navigateur)

React + Vite
Tailwind CSS
Framer Motion

API (Back-End)

Node.js + Express
Middlewares JWT
Controllers / Models

DONNÉES

Supabase / PostgreSQL
+
MongoDB (Logs)

```
{
  "name": "econide-root",
  "version": "1.0.0",
  "description": "Plateforme web de covoiturage écoresponsable",
  "main": "index.js",
  "scripts": {
    "postinstall": "npm install --prefix BackEnd && npm install --prefix FrontEnd",
    "dev": "concurrently \"npm run dev --prefix BackEnd\" \"npm run dev --prefix FrontEnd\"",
    "keywords": [],
    "author": "Thomas COLIN",
    "license": "ISC",
    "devDependencies": {
      "concurrently": "^8.2.2"
    }
  }
}
```

```
services:
  > Run Service
  backend:
    build:
      context: ../BackEnd
      dockerfile: Dockerfile
    ports:
      - "3001:3001"
    env_file:
      - ../BackEnd/.env
    volumes:
      - ../BackEnd:/app
      - /app/node_modules
```

```
PS C:\Prog\Web\EcoRide> npm run dev
[0] [nodemon] watching path(s): *.*
[0] [nodemon] watching extensions: js,mjs,cjs,json
[0] [nodemon] starting `node -r dotenv/config src/server.js`
[1]
[1] VITE v6.4.3 ready in 213 ms
[1]
[1] → Local: http://localhost:5173/
[1] → Network: http://172.31.128.1:5173/
[1] → Network: http://192.168.1.34:5173/
[1] → Network: http://172.31.0.1:5173/
```

Choix technologiques

Front-End

- React + Vite
- Tailwind CSS
- Framer Motion
- React Router v6

Back-End

- Node.js + Express
- JWT (jsonwebtoken)
- bcrypt
- helmet + cors

Bases de données

- Supabase (PostgreSQL)
- MongoDB Atlas (logs)
- Faker.js (seeds)

Infrastructure

- Docker + Docker Compose
- GitHub (versioning)
- Vercel (front)
- Render (back)

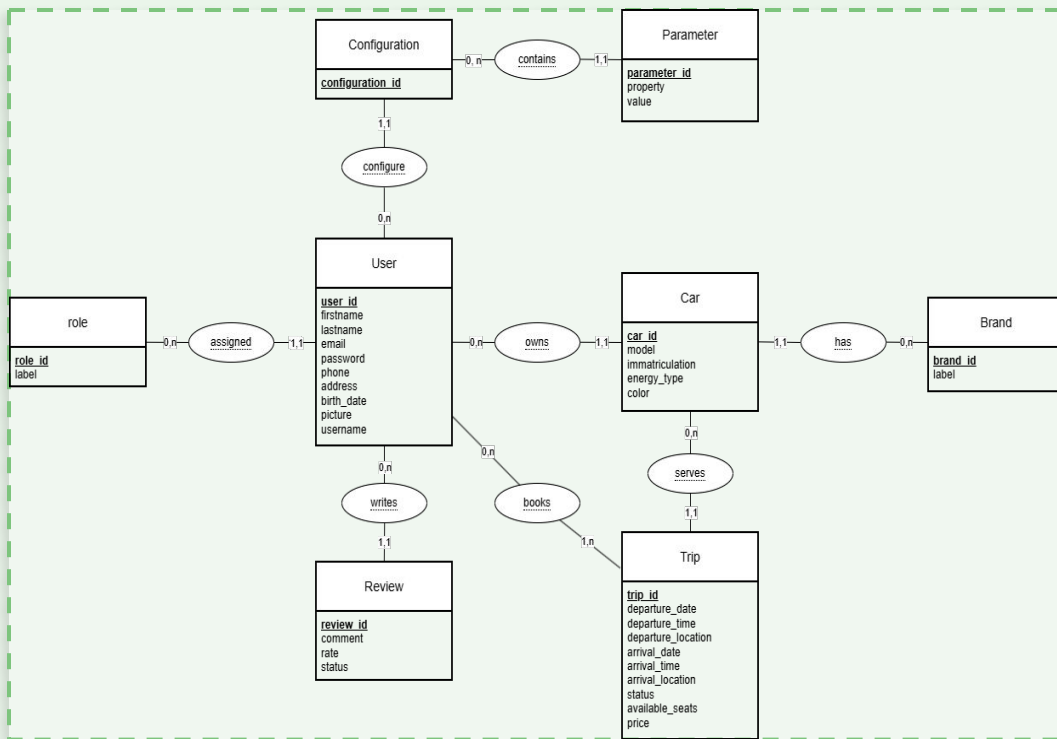
Tests

- Vitest
- Supertest (intégration)

Outils dev

- VS Code
- Postman / Thunder Client
- npm Workspaces
- ESLint

Base de données — Modèle Conceptuel des Données



Tables principales

users id, email, password, role, credits

rides id, driver_id, origin, destination, seats, price

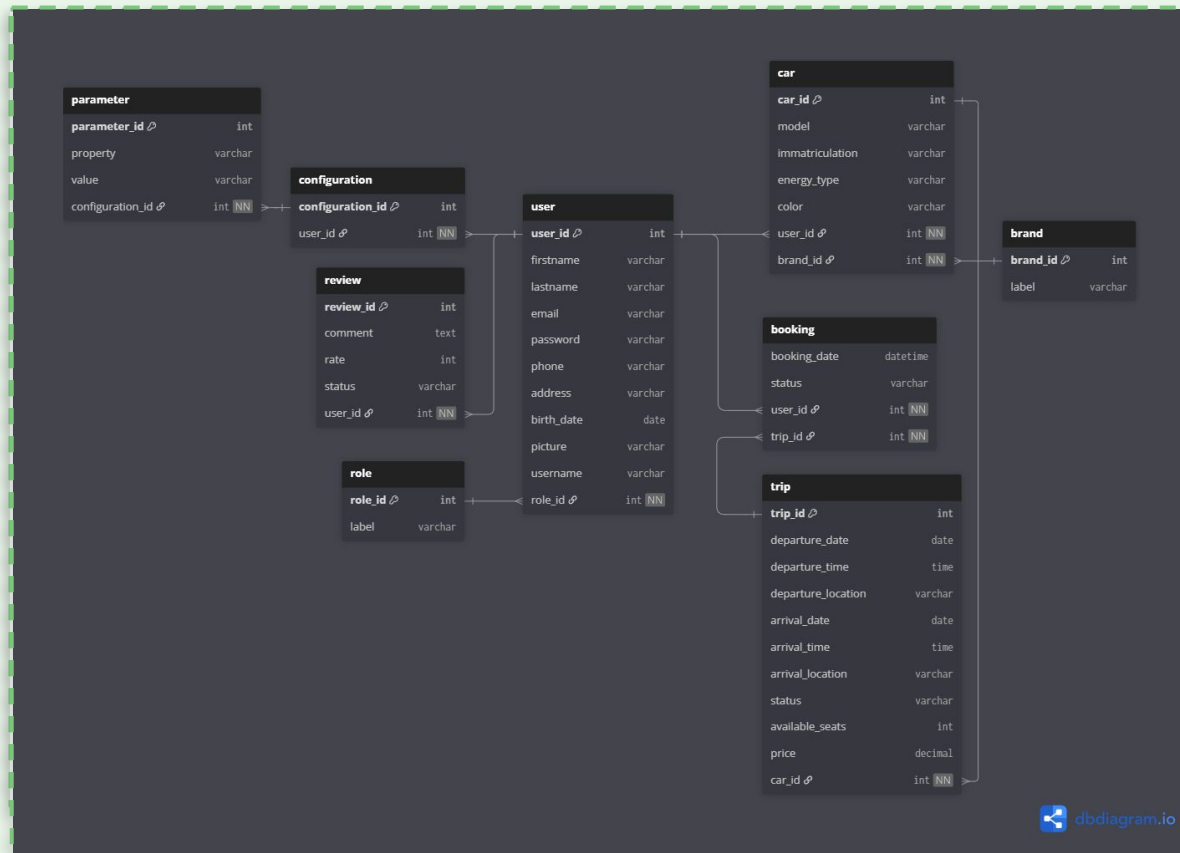
bookings id, ride_id, passenger_id, status

reviews id, booking_id, rating, comment

vehicles id, user_id, brand, model, energy_type

+ MongoDB : collection activity_logs (NoSQL)

Base de données — Modèle Logique des Données



Charte graphique

02 — IDENTITÉ DU LOGO

Logo Principal

Le logo EcoRide associe une feuille stylisée de couleur verte avec le nom de la marque en typographie moderne. L'ensemble symbolise le mouvement, la nature et la technologie.



04 — TYPOGRAPHIE

La typographie d'EcoRide repose sur deux familles complémentaires : une police principale moderne et lisible, et une police secondaire pour les textes courants.

Police Principale — Manrope

Manrope

ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789 !@#\$\$%

Usages

Titre H1 — Bold 36pt
Titre H2 — SemiBold 28pt
Sous-titre — Medium 22pt
Accroche — Regular 18pt

03 — PALETTE DE COULEURS

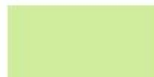
La palette EcoRide est construite autour du vert, couleur de la nature et de l'écologie, associée à des teintes claires et fraîches pour exprimer la modernité et la propreté.



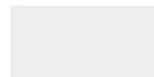
Vert Forêt
#006A4E
RVB 0 106 78



Vert Citron
#78C850
RVB 120 200 80



Vert Pâle
#D0ED9D
RVB 208 237 157



Blanc
#FFFFFF
RVB 255 255 255



Bleu Brume
#E1F1F8
RVB 225 241 248

07 — VOIX & TON DE LA MARQUE

La voix d'EcoRide est le reflet de ses valeurs : tournée vers l'avenir, propre dans ses messages, fiable dans ses engagements et profondément écologique dans sa vision.

Les 4 Piliers de la Voix EcoRide



Avant-gardiste

Nous parlons de futur, d'innovation et de progrès.



Épuré

Des messages directs, sans jargon ni surcharge.



Fiable

Des engagements tenus, une communication transparente.



Écologique

La durabilité est au cœur de chaque message.

Développement Front-End

SPA React + Vite

Démarrage instantané, HMR, build optimisé

Context API + JWT

État auth global, interception Axios, logout auto

React Router v6

Routes publiques vs protégées, redirection par rôle

Tailwind CSS

Palette personnalisée, classes utilitaires, purge prod

Framer Motion

Transitions de page, animations entrée des cartes

```
import { createContext } from 'react';
import { useAuthLogic } from "../hooks/auth/useAuthLogic";

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const authData = useAuthLogic();

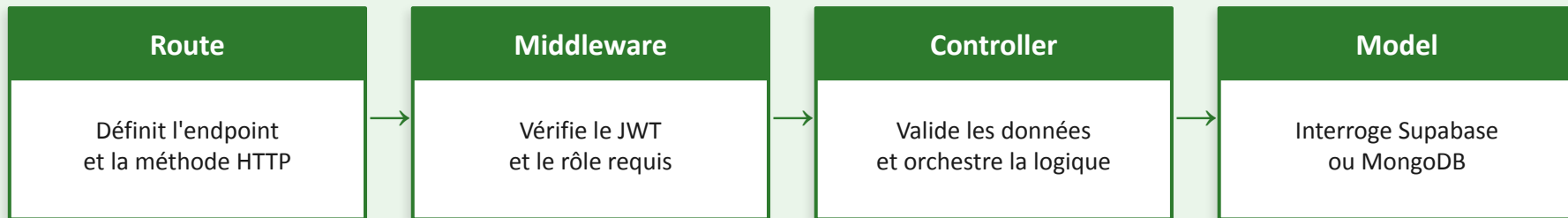
  return (
    <AuthContext.Provider value={authData}>
      {children}
    </AuthContext.Provider>
  );
}

export { AuthContext, AuthProvider };
```

```
const isTokenExpired = (token) => {
  if (!token || token === 'undefined') return true;
  try {
    const base64Url = token.split('.')[1];
    const base64 = base64Url.replace(/-/g, '+').replace(/_/g, '/');
    const jsonPayload = decodeURIComponent(
      atob(base64)
        .split('')
        .map((c) => '%' + ('00' + c.charCodeAt(0).toString(16)).slice(-2))
        .join('')
    );
    const { exp } = JSON.parse(jsonPayload);
    return Date.now() >= exp * 1000;
  } catch (e) {
    return true;
  }
};
```

Développement Back-End — API REST Express

Sécurité : helmet (headers HTTP), CORS restreint, validation des entrées, mots de passe hashés avec bcrypt (coût 12)



```
import jwt from 'jsonwebtoken';

export default function authMiddleware (req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(401).json({ error: 'Access denied: No token provided.' });
  }

  const parts = authHeader.split(' ');
  if (parts.length !== 2 || parts[0] !== 'Bearer') {
    return res.status(401).json({ error: 'Invalid token format (Bearer required).' });
  }

  const token = parts[1];

  jwt.verify(token, process.env.JWT_SECRET || process.env.SECRET_KEY, (err, userPayload) => {
    if (err) {
      return res.status(403).json({ error: 'Session expired or invalid token.' });
    }

    req.userId = userPayload.id;
    next();
  });
};
```

```
const login = async (req, res) => {
  try {
    const { identifier, password } = req.body;

    await validateLoginInput(identifier, password);

    let user;
    if (isValidEmail(identifier)) {
      user = await User.findByEmail(identifier);
    } else {
      user = await User.findByUsername(identifier);
    }

    if (!user) {
      throw new AuthenticationError('Invalid credentials');
    }

    await verifyPassword(password, user.password);
    const response = handleLoginSuccess(user);
    return res.status(200).json(response);
  }
};
```

MongoDB — Journalisation des logs admin

Pourquoi MongoDB pour les logs ?

- Schéma flexible : chaque action peut avoir des attributs différents
- Écriture rapide : les logs sont écrits en masse, la cohérence transactionnelle n'est pas prioritaire
- Complémentarité avec PostgreSQL : séparation claire des responsabilités (données métier vs traçabilité)

Initialisation :

```
import mongoose from 'mongoose';

export const connectMongo = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
  } catch (err) {
    throw new Error(`Connexion MongoDB impossible : ${err.message}`);
  }
};
```

Utilisation :

```
import mongoose from 'mongoose';

const logSchema = new mongoose.Schema({
  userId: String,
  action: String,
  details: mongoose.Schema.Types.Mixed,
  createdAt: { type: Date, default: Date.now }
});

const Log = mongoose.model("Log", logSchema);
export default Log;
```

Chaque action admin (suspension, suppression, modification de crédits) déclenche un `writeLog()` asynchrone, sans bloquer la réponse API.

Tests — Vitest

2

suites de tests

7

tests passés

0.9s

durée d'exécution

Tests unitaires

- Middleware authenticate (401, 200, 403)
- Fonction validateRegister (email, password)
- Utilitaires de formatage et validation

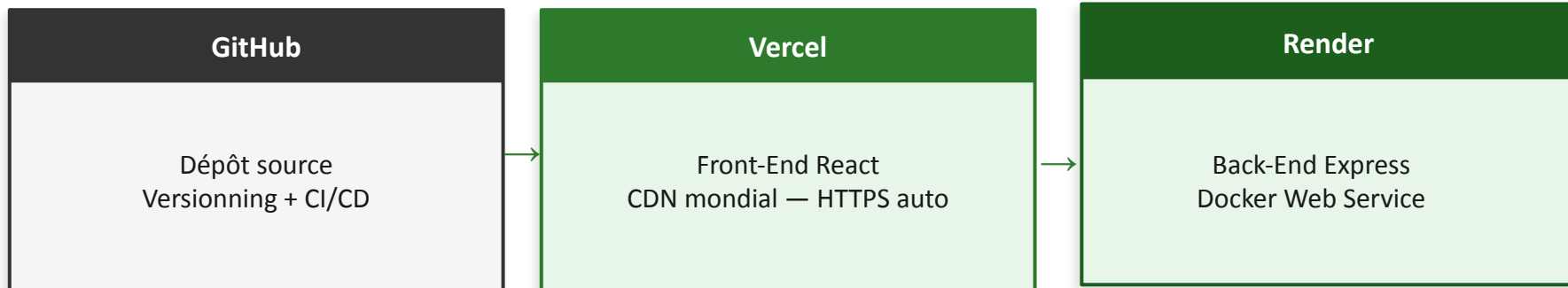
Tests d'intégration (prévus)

- POST /api/auth/register → 201
- POST /api/auth/login → token JWT
- GET /api/rides → liste trajets filtrée

```
async function userMiddleware(req, res, next) {
  try {
    const userId = req.user?.id;
    if (!userId) {
      return res.status(401).json({ error: 'Utilisateur non authentifié' });
    }

    const user = await User.findById(userId);
    if (!user) {
      return res.status(401).json({ error: 'Utilisateur introuvable' });
    }
    req.user = user;
    next();
  } catch (err) {
    next(err);
  }
}
```

Déploiement et infrastructure



Docker — docker-compose.yml

- Service backend (Node.js) : PORT 3001, variables d'env injectées
- Service frontend (Nginx) : PORT 5173:80, depends_on backend
- Image multi-étapes (alpine) : builder → prod, taille minimale

```
✓ built in 2.75s
Build Completed in /vercel/output [4s]
Deploying outputs...
Deployment completed
```

```
==> Your service is live 🎉
==>
==> //////////////////////////////////////
==>
==> Available at your primary URL https://ecoride-backend-tx09.onrender.com
```

Difficultés rencontrées et solutions

Architecture isolée front / back

Problème : Gérer deux serveurs de développement, les ports, le CORS et les variables d'environnement de façon cohérente.

Solution : Monorepo + Concurrently pour lancer les deux apps simultanément. Docker Compose pour unifier l'infrastructure.

Authentification JWT sécurisée

Problème : Vérifier le token sur chaque requête, gérer l'expiration, protéger les routes par rôle sans duplication de code.

Solution : Middleware authenticate() réutilisable + authorize(...roles) composables sur toutes les routes sensibles.

Synchronisation SQL + NoSQL

Problème : Utiliser deux bases de données de nature différente sans coupler leurs logiques ni créer de latence.

Solution : Écriture des logs MongoDB en asynchrone (non bloquant) après chaque action admin, entièrement découplée de PostgreSQL.

Bilan et perspectives

Ce que ce projet m'a apporté

- Maîtrise du cycle complet : conception → prod
- Architecture full-stack découplée et scalable
- Sécurisation d'une API REST (JWT, rôles, bcrypt)
- Gestion de deux BDD hétérogènes (SQL + NoSQL)
- Écriture de tests automatisés (TDD)
- Containerisation Docker + CI/CD GitHub
- Rigueur de documentation (README, variables d'env)

Prochaines étapes prévues

- Notifications en temps réel (WebSockets / Socket.io)
- Intégration paiement (Stripe API)
- Application mobile (React Native)
- Cookies HTTPOnly à la place du localStorage
- Tests end-to-end (Playwright ou Cypress)
- PWA pour usage hors ligne

Ce projet constitue une base solide et extensible, prête à évoluer vers un produit réel.

Conclusion

EcoRide couvre l'ensemble du référentiel DWWM :

interfaces dynamiques, API REST sécurisée, bases de données, déploiement.

- Front-End React moderne (Vite, Tailwind, Framer Motion, Context API)
- Back-End Express sécurisé (JWT, rôles, bcrypt, helmet)
- Double BDD : PostgreSQL (Supabase) + MongoDB (logs)
- Infrastructure Docker + déploiement Vercel / Render
- Tests automatisés Vitest



Des questions ?

Thomas Colin · thomas-colin@hotmail.com · GitHub : github.com/thomas-colin